

## 35.8. Deux BONHOMME, un seul dessin : le modèle, comme une classe

Un Perso est un modèle : `montrerHeros(numero, x, y)` en pose deux exemplaires, chacun avec ses lutins, sans copier le dessin.

### 1. Ce qu'il faut comprendre

**Ce qui est nouveau ici :** `montrerHeros` reçoit un `numero`, le premier lutin à employer. Avec lui, on pose **deux** BONHOMME : `montrerHeros(0, 40, 64)` prend les lutins 0 à 3, `montrerHeros(4, 104, 64)` les lutins 4 à 7.

**Un seul dessin, deux exemplaires.** BONHOMME n'est écrit qu'une fois, et ses quatre tuiles ne sont gravées qu'une fois dans la cartouche. Chaque `sprite16` en fait un **exemplaire** à l'écran, avec sa place à lui. C'est l'idée d'une **classe** : le modèle (BONHOMME) et ses objets (les deux à l'écran).

**Pourquoi 4 et pas 1 :** un personnage de  $16 \times 16$  occupe **quatre** lutins. Le second doit donc commencer après les quatre du premier : 0, 1, 2, 3 pour l'un, 4, 5, 6, 7 pour l'autre. Avec `montrerHeros(1, ...)`, les deux se partageraient trois lutins, et l'un écraserait l'autre.

**Déroulé de `montrerHeros(4, 104, 64)` :** `numero` vaut 4, `x` 104, `y` 64, puis `sprite16(4, 104, 64, BONHOMME)` pose les lutins 4 (104, 64), 5 (112, 64), 6 (104, 72) et 7 (112, 72).

#### Les fonctions de cette leçon — et le tuto de chacune

- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)
- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de  \$16 \times 16\$](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **488 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          47 o ← la console, à l'allumage — la mise en route de la console
montrerHeros()     219 o ← main() — ta fonction
main()             39 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
RangerLutins       11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles       31 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
Tuiles            64 o ← CopierTuiles — 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert   10 o ← InstallerTransfert — la petite routine qui copie les lutins
Lettres gravées : aucune
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond,
EffacerPanneau, LireManette, Notes
```

```
Fonctions du langage appelées :
sprite()      x 4   218 o sur place, dans montrerHeros()
image()       x 1    6 o sur place, dans main() + AttendreImage (28 o)
sprite16()    x 1    0 o sur place, dans montrerHeros()
Tes fonctions appelées :
montrerHeros() x 2   29 o sur place, dans main()
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Deux exemplaires du même modèle : le dessin n'est écrit qu'une fois.

#include "personnages.cpp" // la liste : BONHOMME
#include "heros.cpp"       // montrerHeros(numero, x, y)
```

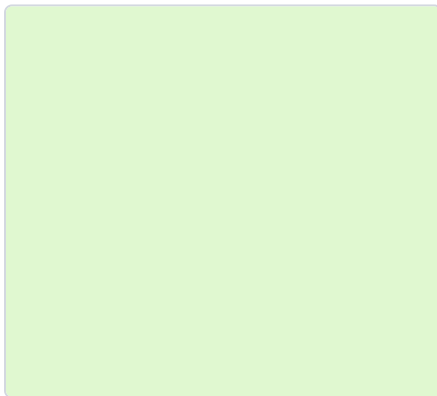
## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    montrerHeros(0, 40, 64); // le premier : lutins 0 à 3, en (40, 64)
    montrerHeros(4, 104, 64); // le second : lutins 4 à 7, en (104, 64)

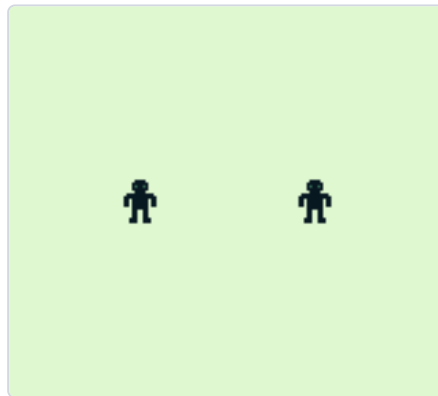
    while (true) {
        image();
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

Deux bonshommes côte à côte, le même dessin.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le premier en (40, 64) (40, 64)

✓ le second en (104, 64) (104, 64)

✓ le même dessin : les mêmes tuiles

---

Ensuite : **35.9. BONHOMME qui marche : le faire bouger depuis principal.cpp** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++